STUDY GROUP ON FUTURE DATA INFRASTRUCTURE (SG-FIT)

Move Less – Use More: tech demo

DEMO SCRIPT https://github.com/wmo-im/metdx-demo/blob/main/docs/demo/infcom4-sg-fit-side-event-2026-11-25.md

WORLD METEOROLOGICAL ORGANIZATION

Step 1 — Find the dataset

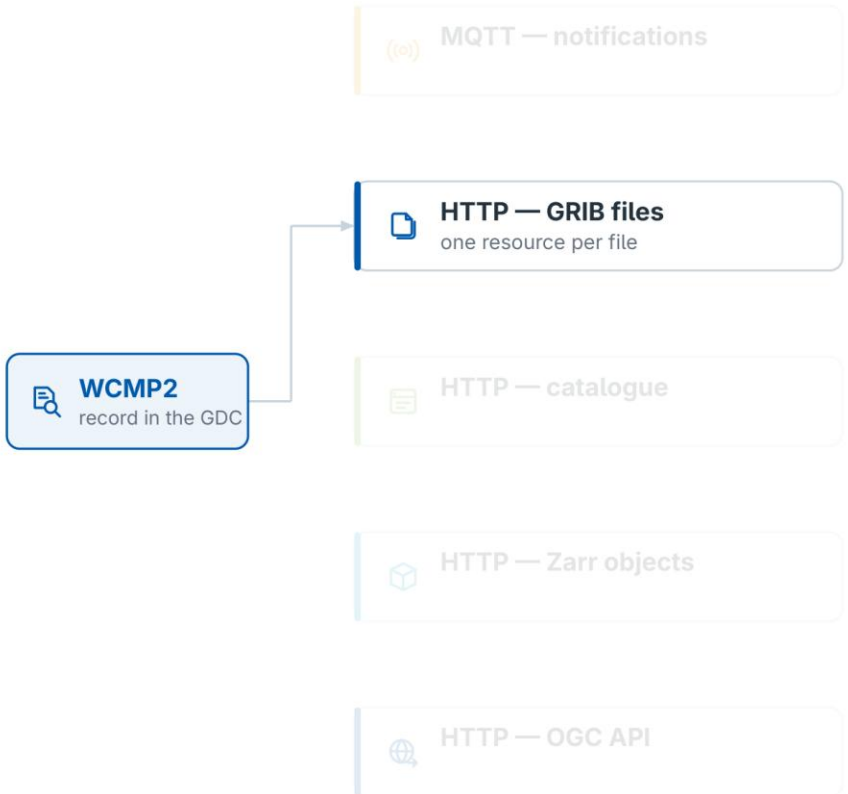
Discovery metadata is the entry point to everything that follows



- A **WCMP2** discovery metadata record is published to the WIS2 **Global Discovery Catalogue** — an OGC API – Records service.
- It gives a user enough to *find* the dataset and decide whether it meets their requirements.
- It also says how to get the data: download, API, or subscribe.
- If it is not in the catalogue, users cannot find it.

Step 2 — The data is a pile of GRIB files

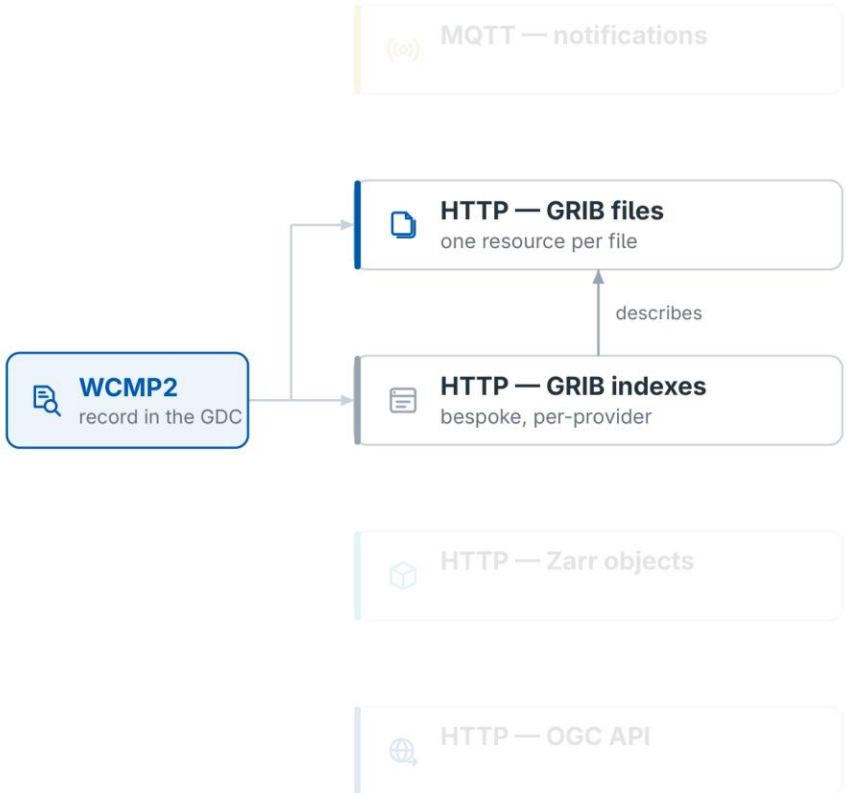
This is, after all, WMO



- WIS2 is Web-oriented, so the GRIB files are published as **HTTP resources**. The WCMP2 record may point at a Web Accessible Folder.
- WIS2 does *not* formalise how resources are grouped or organised, and does not require a file-naming convention.
- So the user has to navigate whatever the data provider chose.
- And a single dataset can be thousands of individual resources to manage: a significant overhead.

Step 3 — Which file do I actually need?

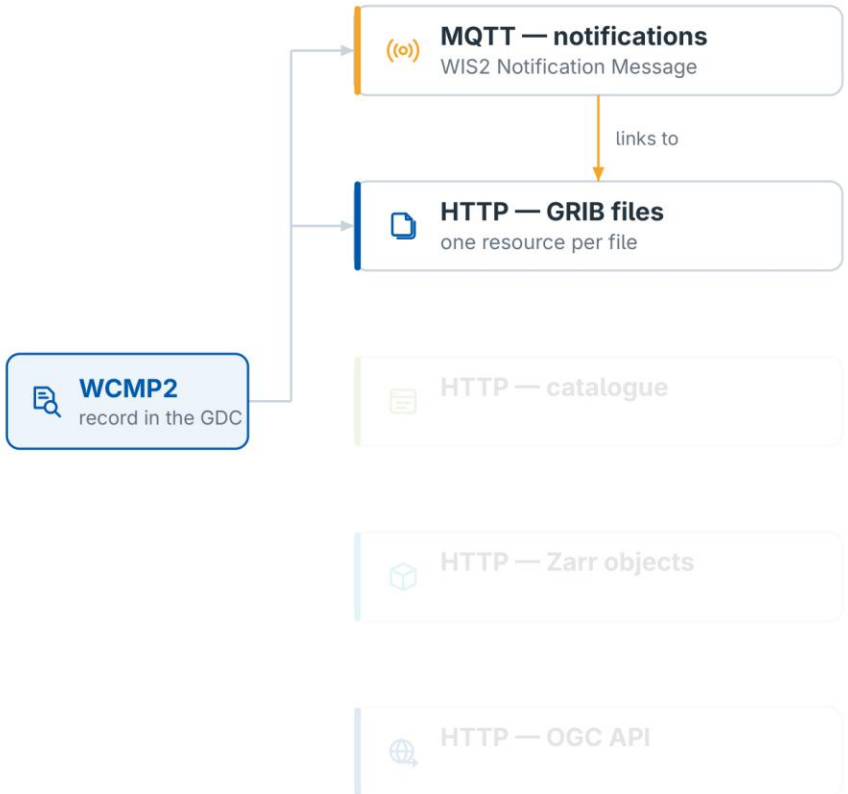
The metadata is inside the GRIB, which is the wrong place for it



- To know what a file contains you must open and parse it. Fine on local POSIX storage; painful for objects on cloud storage.
- **ECMWF** and **NOAA** both recognised this and independently adopted data models, tooling and services that publish *indexes* describing each GRIB.
- It works — but it is not standardised. Different tooling, different formats, different semantics.

Step 4 — Real-time: publish / subscribe

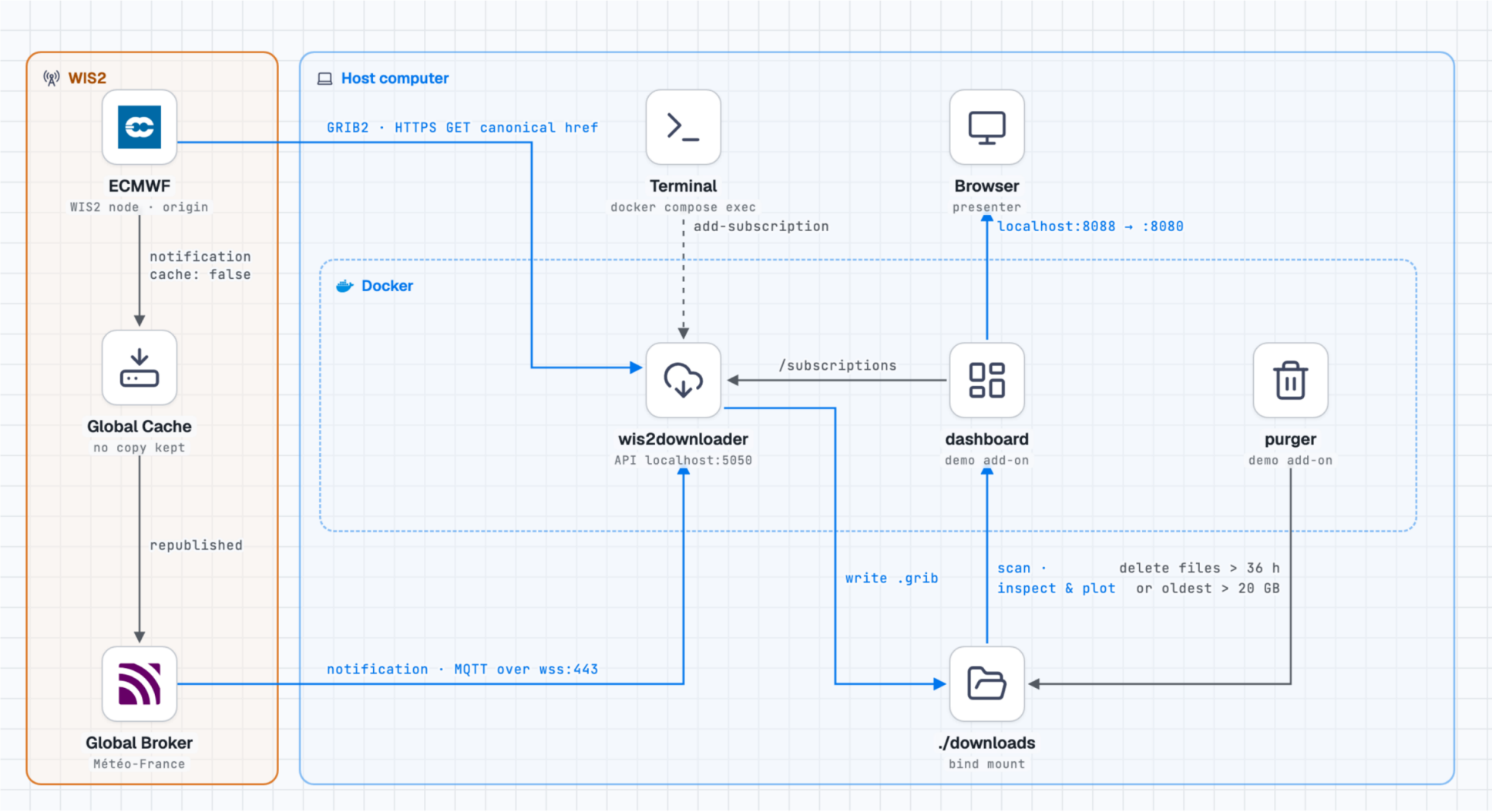
WIS2 replaces push-by-routing-table with subscribe-to-what-you-need



- WIS2 adds **Publish-Subscribe (Pub/Sub)** over MQTT, with a standard Topic Hierarchy and a GeoJSON Notification Message.
- Each notification message describes a newly available resource — date, time, location, topic — and links to the data itself.
- The WIS2 Cookbook has a recipe for richer metadata to support **client-side filtering**, and for declaring those filters in WCMP2.
- Users subscribe to the topics they care about and download only what they need.

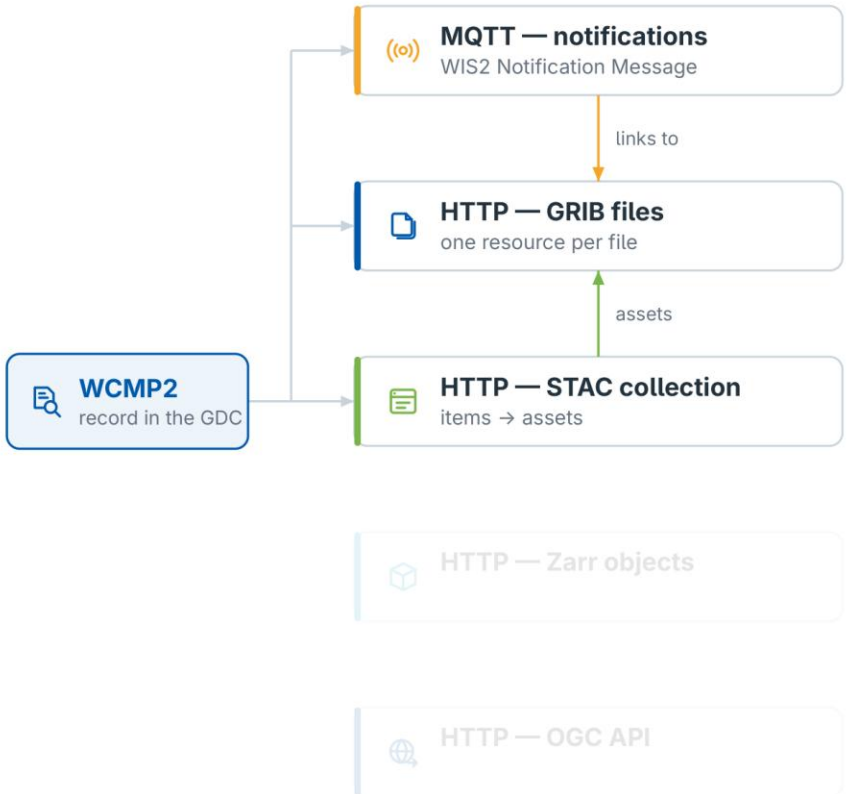
Step 4 — In practice: wis2downloader

Subscribe to a topic, download each file as it's announced



Step 5 — Standardise the index: STAC

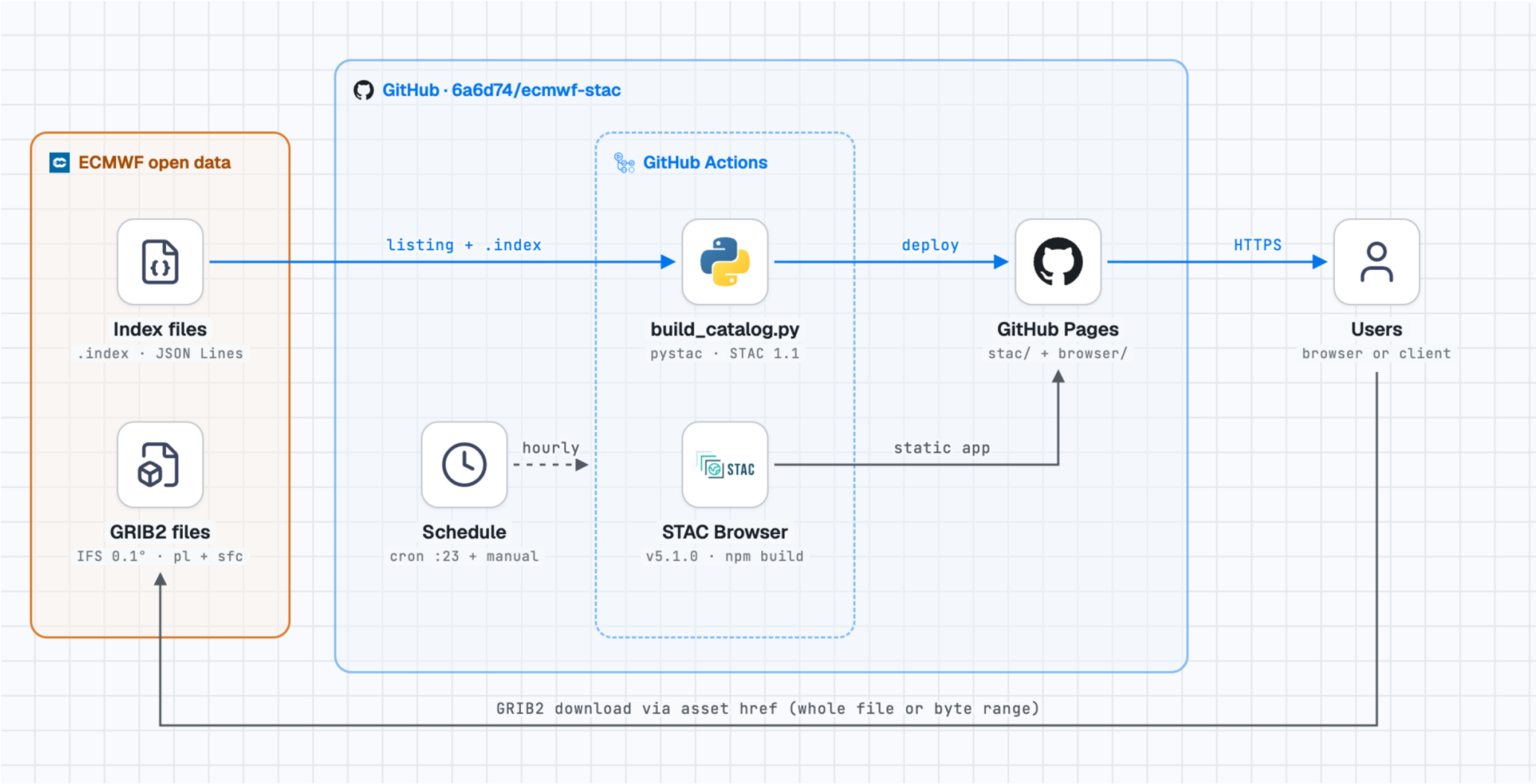
Make a pile of files easy to navigate



- STAC — the SpatioTemporal Asset Catalog, an OGC community standard — is a common way to browse collections of geospatial data resources, whatever format the data is in.
- Collections organise data into virtual folders; items carry resource-level metadata and point at **assets** — the data resources themselves.
- Wide adoption and good open-source tooling. Providers could publish STAC alongside their resources.
- We've built one for ECMWF's open data, generated hourly from ECMWF's index files, without downloading a GRIB.

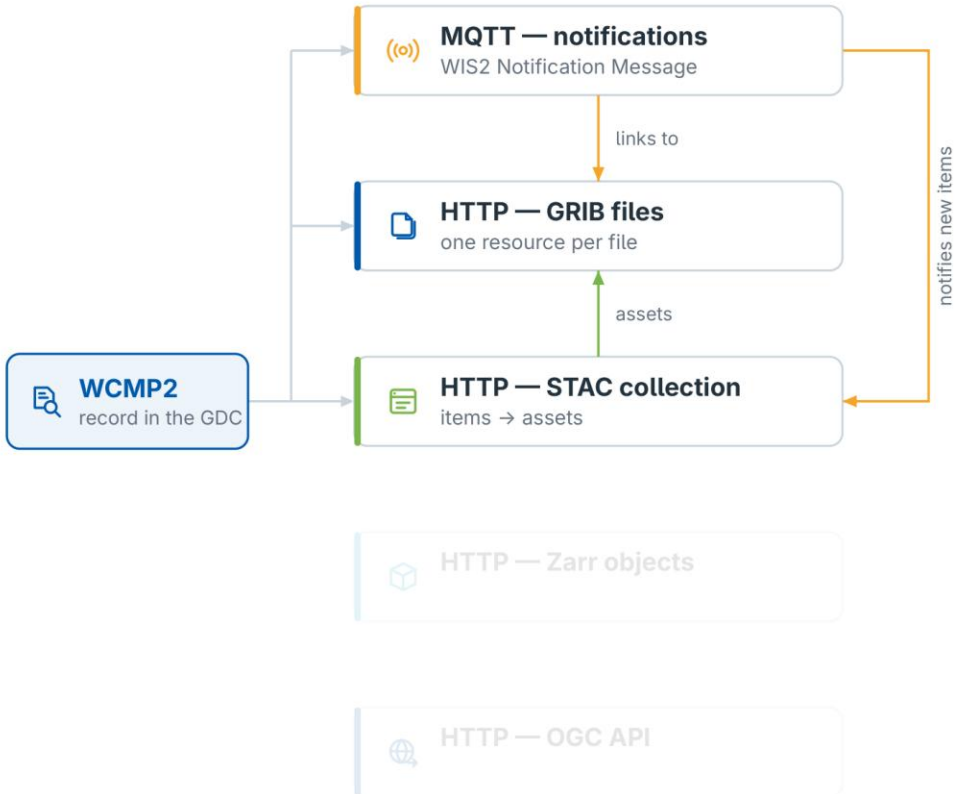
Step 5 — In practice: a STAC catalogue for ECMWF open data

Built hourly from ECMWF's index files and published on GitHub Pages



Step 6 — One message per file is a lot of messages

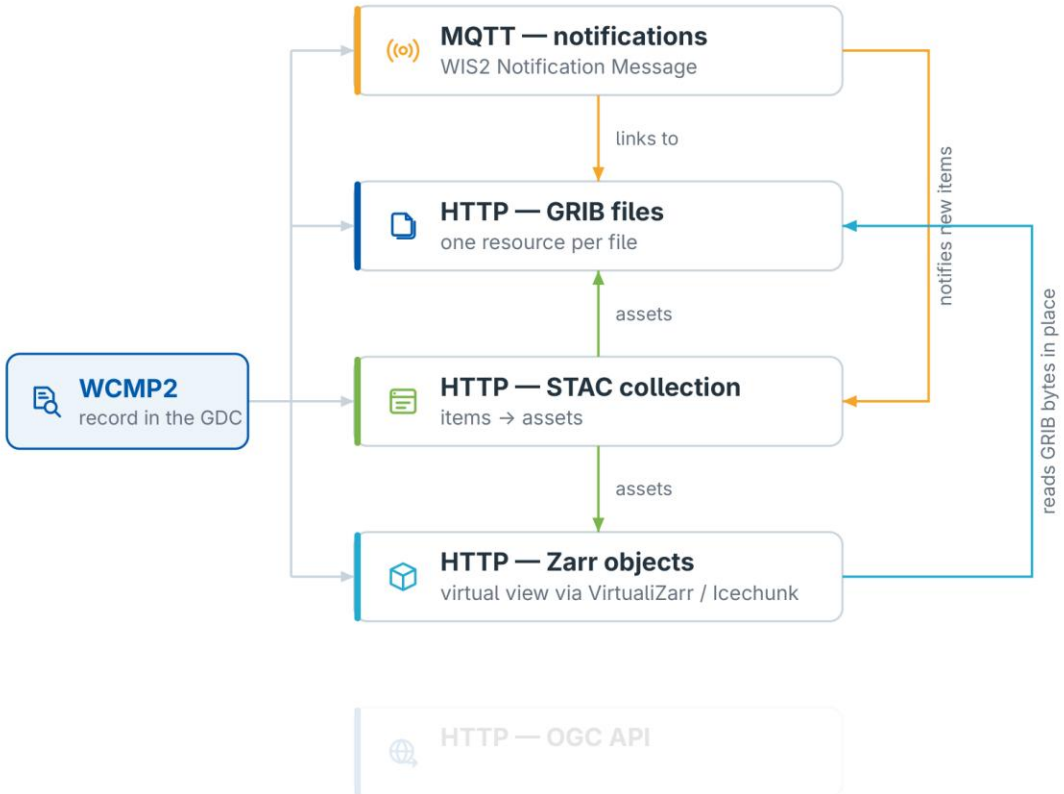
A notification per model run, not per file



- One WIS2 Notification Message per GRIB file. For a large model run, that is a great many messages for a user to receive and filter.
- With a standard index, one message could announce a new STAC item, such as a whole model run, and the user follows it to the GRIBs they need.
- For early access, republish the item with "rel": "update" as more files arrive.

Step 7 — Work at the dataset level: virtual Zarr

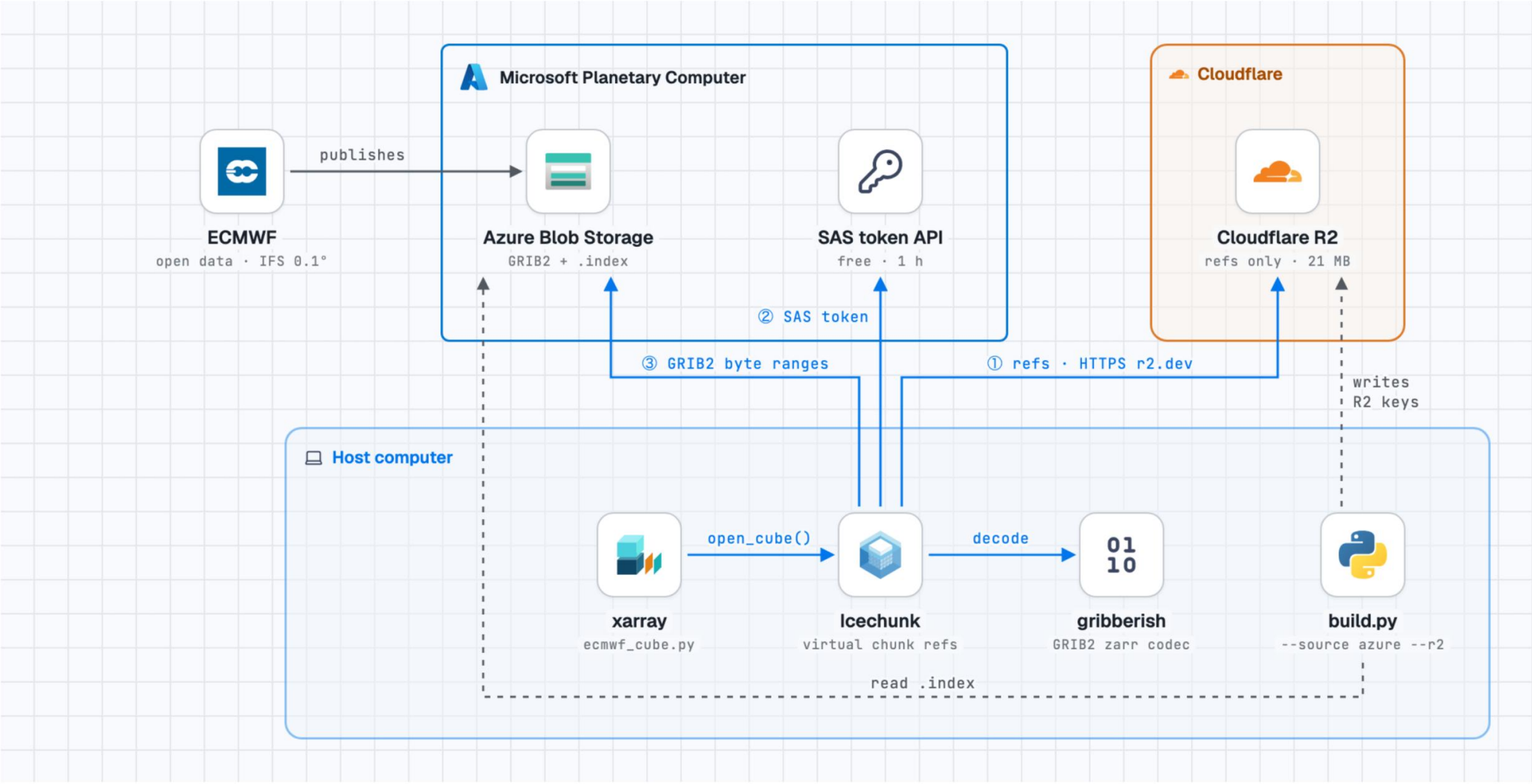
Let the software figure out which bytes a calculation needs



- Users would rather not handle thousands of resources. Cloud-optimised formats such as Zarr let you open a whole dataset lazily: `ds = xarray.open_zarr(…)` — bytes are fetched only when needed.
- Native Zarr is one option. Given WMO's continuing need for legacy formats, VirtualiZarr / Icechunk are attractive: the data stays in its native format, the user sees a Zarr store, and an index says which bytes to read.
- STAC collections can carry Zarr assets alongside GRIB assets, so users find the store they need.
- We've built one: an Icechunk cube of ECMWF's 0.1° forecasts; the GRIBs stay where ECMWF publishes them.

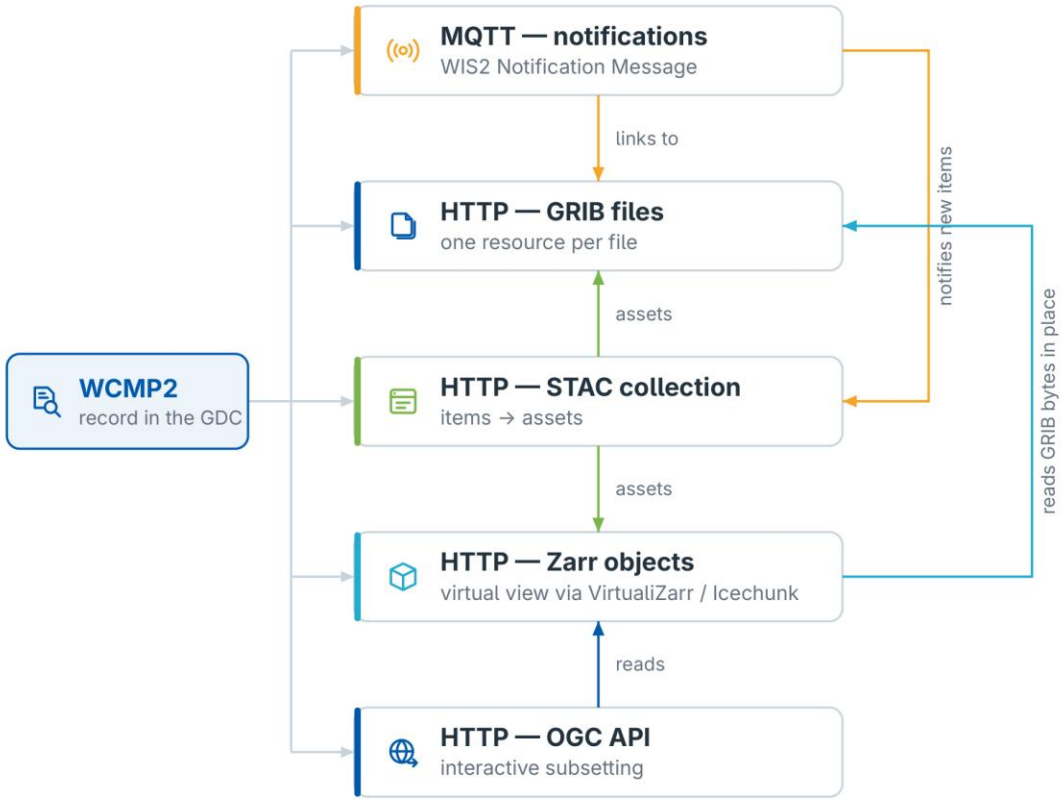
Step 7 — In practice: a virtual Zarr cube

Icechunk references on Cloudflare R2; the GRIB bytes stay on Azure



Step 8 — For the rest of us, a Web API

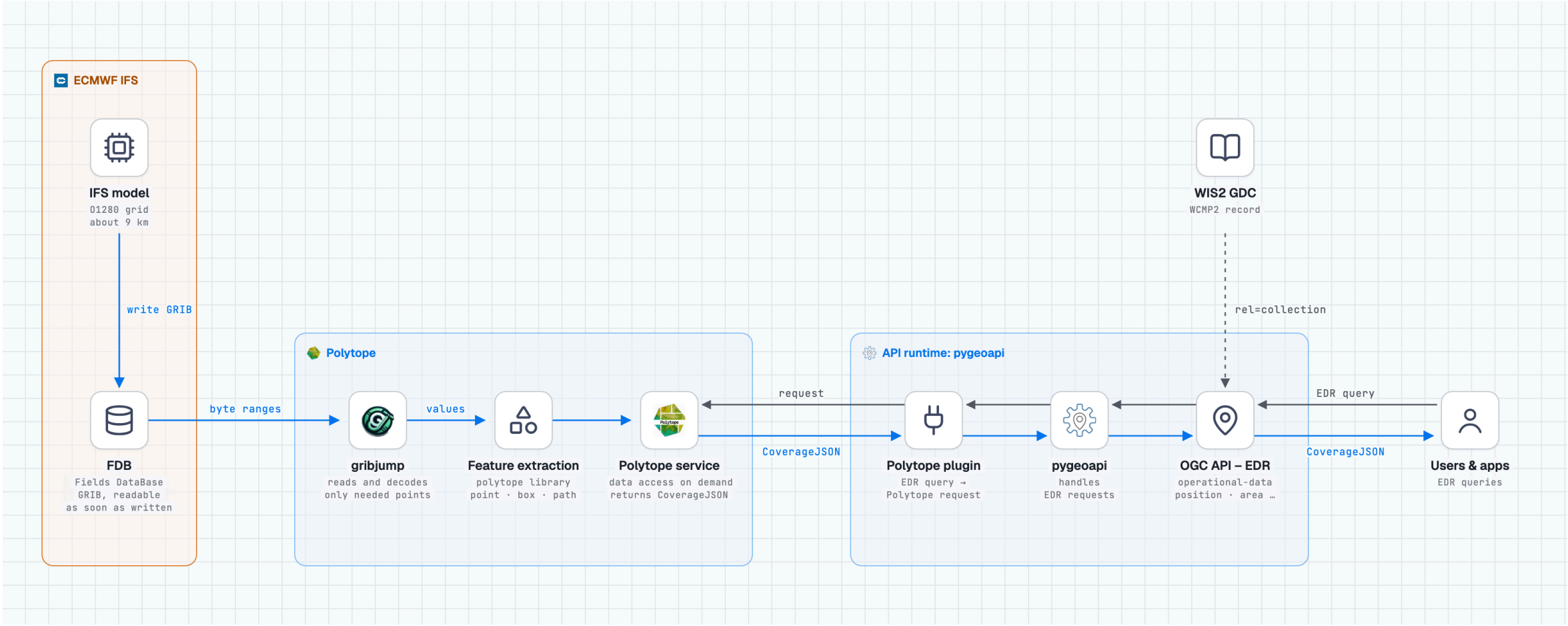
Most users want a small subset/extract or graph/image



- Zarr suits power users willing to learn how a dataset is structured. Many users just want a snippet for a Web application.
- An HTTP **GET** meets that need: the subset, in the format they asked for.
- EUMETNET MeteoGate already demonstrates this — interactive Web services on WIS2 foundations.
- How the request is served is the provider's choice, and there is always a cost — especially at scale. Cloud-optimised data is a good foundation. There are many options to implement an OGC API: pygeoapi (Xarray by default); pygeoapi with custom tooling (ECMWF Polytope and Earthkit); IBL; ...
- We've built a Web UI to demonstrate how user applications can interact with Web APIs in a metadata-driven workflow.

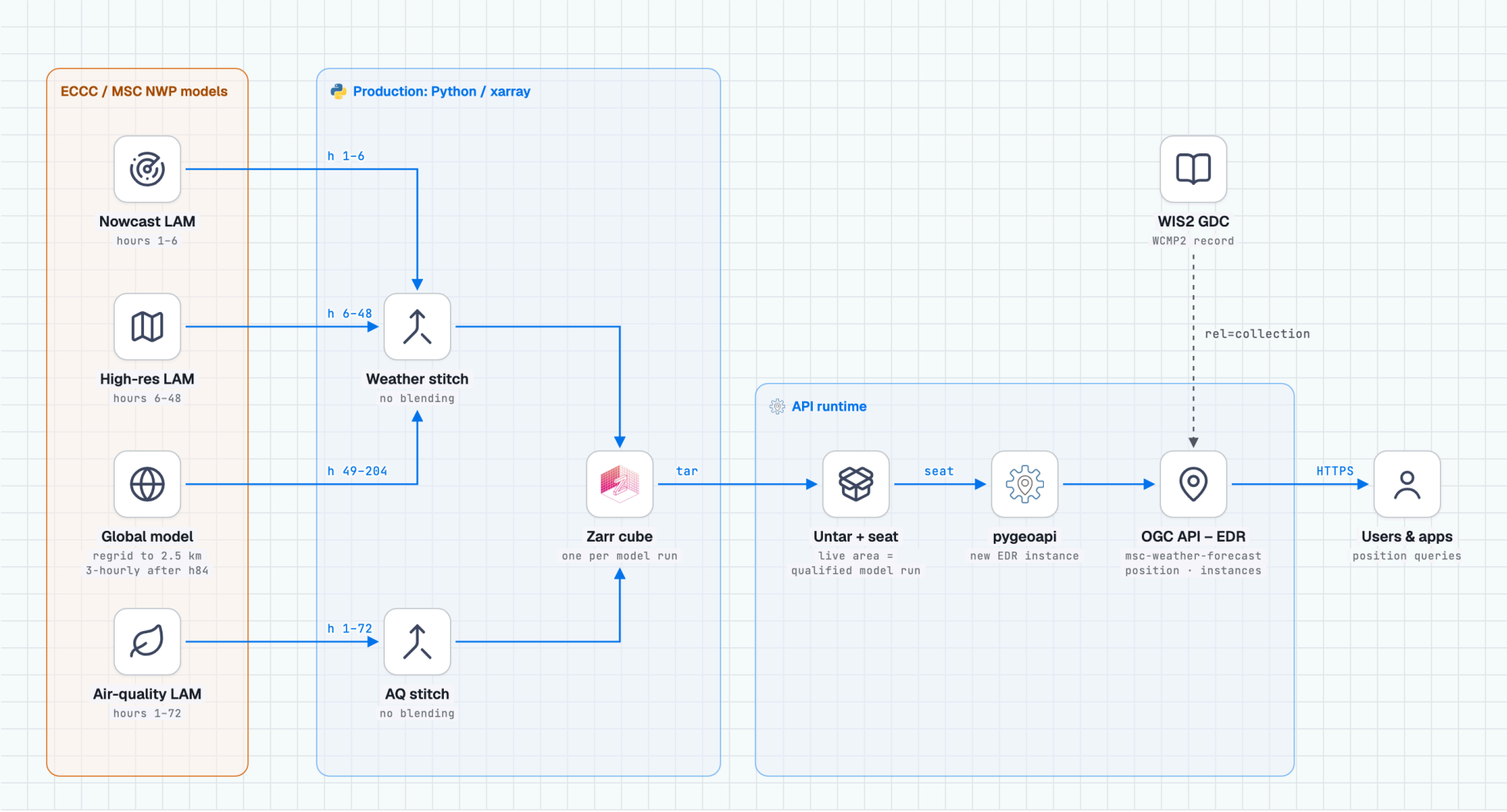
Step 8 — ECMWF: OGC API – EDR over Polytope

Reads only the values needed, straight from the FDB



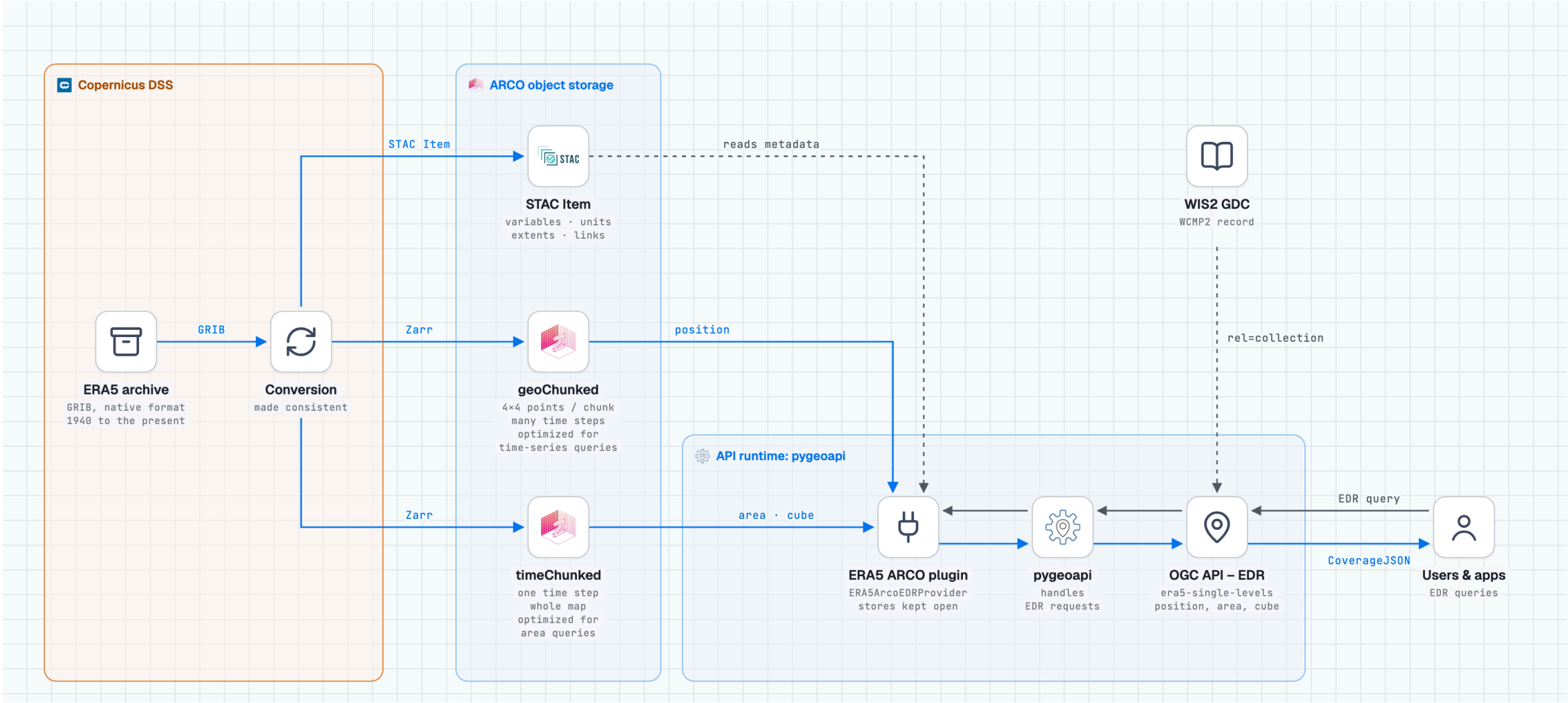
Step 8 — ECCC: a stitched forecast cube behind OGC API – EDR

Four models stitched into one Zarr cube per run, served by pygeoapi



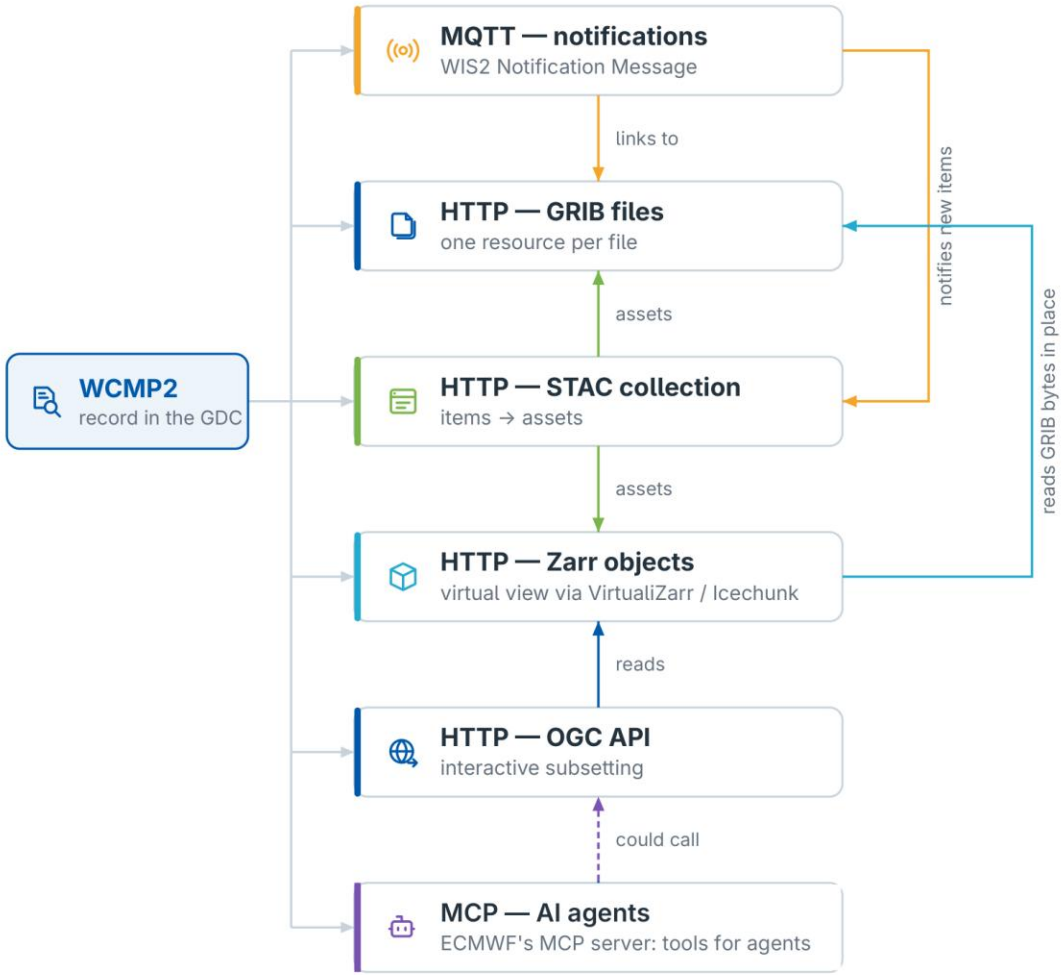
Step 8 — ERA5: OGC API – EDR over ARCO Zarr

Two copies, one chunked for time series and one for maps



And beyond — integration with AI agents

Let an AI agent follow the links for you



- AI agents can follow the same metadata-driven workflow. The WCMP2 record links to ECMWF's MCP server: the Model Context Protocol, an open standard for agents to discover and use a provider's tools.
- An agent such as Claude finds the link in the record, connects, and answers in plain language: "Show me the ECMWF meteogram for Geneva."
- Today the server returns ECMWF's charts, maps and catalogue information. Next: agents requesting data through OGC APIs such as EDR.